

Rappels de la semaine précédente

Python pour la production

IDE

- Environnement de développement intégré
- Vous permet d'éditer du code à l'aide d'outils qui vous facilitent la vie:
 - linter
 - formatteur
 - debugger
 - extensions personnalisées
- Intégration des LLMs -> possibilité d'étudier votre codebase, découvrir des erreurs, ajouter de la doc, etc.
- Si utilisation d'un provider externe (OpenAI, Anthropic, etc.), attention à la confidentialité de votre code ⚠

Python

- Langage de programmation interprété, simple d'utilisation et très répandu aujourd'hui
- Permet d'utiliser des bibliothèques écrites dans d'autres langages (C++, Rust, Go, etc.):
`pandas` , `polars` , `numpy` , `torch` , ...
- Cela permet de compenser les lenteurs inhérentes à Python
- Aujourd'hui, version stable 3.14, maintenue au moins jusqu'en 2030
- Le choix de la version de python dépend de vos besoins et du projet dans lequel vous vous intégrez
- Lorsqu'on commence un projet, on travaille avec des environnements virtuels qui vont venir isoler nos installations de bibliothèques pour éviter de polluer d'autres projets sur votre machine → par exemple via la commande `uv venv -p 3.14`

- Un projet complet contient à minima les fichiers `README.md`, `pyproject.toml`, des tests, ainsi qu'une arborescence contenant votre code python

```
├── README.md
├── pyproject.toml
├── src/
│   ├── my_project/
│   │   ├── __init__.py
│   │   ├── main.py
│   │   ├── core/
│   │   │   ├── __init__.py
│   │   │   └── engine.py
│   │   └── utils/
│   │       ├── __init__.py
│   │       └── helpers.py
└── tests/
    ├── __init__.py
    ├── test_core.py
    └── test_utils.py
```

- Les tests sont essentiels pour assurer que vos **modifications futures ne perturbent pas le bon fonctionnement de votre code déjà existant** -> besoin de faire des tests unitaires qui viennent tester une partie précise de votre code
- Des librairies python comme `pytest` permettent de lancer vos tests, d'avoir des statistiques sur la proportion de votre code qui a réellement été testé
- Les tests sont à lancer soit manuellement soit automatiquement après chaque modification de votre code

Debug

- Les IDEs proposent tous des fonctionnalités d'aide au débogage
- **Points d'arrêts** (breakpoints) permettent de s'arrêter à un moment précis de votre code afin d'inspecter des variables, vérifier que l'état est conforme à vos attentes:
 - *point d'arrêt classique*: le code s'arrête à l'endroit du breakpoint à chaque exécution de la ligne
 - *point d'arrêt conditionnel*: on s'arrête uniquement si une condition est vérifiée (souvent utile dans des boucles `for`, `while`, etc.)
 - *point d'arrêt sur les exceptions levées*: utile pour les tests, si votre code casse et que vous voulez directement aller au moment de l'erreur pour inspecter ce qui ne va pas

Gestion des dépendances

- Dans votre `pyproject.toml`, vous définissez des dépendances de votre projet afin d'utiliser des librairies externes
- Plusieurs opérateurs permettent de spécifier les versions des librairies que vous voulez installer:
 - `>`, `>=`, `<`, `<=`, `~=`, `===`, `!=`
 - la séparation par une virgule permet de combiner des conditions (ex: `numpy >=2.0.0, <3.0.0`)
- Si vous souhaitez installer les dépendances d'un projet sur lequel vous venez d'arriver: `uv sync`

Fonctions utiles en python

lambda function

```
f = lambda x, y=1: x + y  
print(f(5,6))  # ?  
print(f(5))    # ?
```

map + lambda function

```
my_list = [1, 2, 3, 4, 5, 6]  
print(list(map(lambda x: x + 1, my_list)))  
# ?
```

Fonctions utiles en python

lambda function

```
f = lambda x, y=1: x + y
print(f(5,6)) # 11
print(f(5)) # 6
```

map + lambda function

```
my_list = [1, 2, 3, 4, 5, 6]
print(list(map(lambda x: x + 1, my_list)))
# ?
```

Fonctions utiles en python

lambda function

```
f = lambda x, y=1: x + y  
print(f(5,6)) # 11  
print(f(5)) # 6
```

map + lambda function

```
my_list = [1, 2, 3, 4, 5, 6]  
print(list(map(lambda x: x + 1, my_list)))  
# affiche: [2, 3, 4, 5, 6, 7]
```

Git

- Toutes ces bonnes pratiques permettent d'avoir une base de travail commune pour permettre un travail collaboratif
- Afin de partager votre code et de contribuer sur un projet, des outils de gestion de version comme `Git` permettent de travailler à plusieurs
- `git init` pour initialiser un projet, `git add` pour ajouter des modifications à mon projet et `git commit` pour les valider
- Si je développe une nouvelle feature, je crée une branche avec `git switch -c ma_super_branche`
- Pour travailler avec des repos distants: `git clone`, `git fetch`, `git pull`